

4. Methods

- ❖ Method provides the functionality or operation or behaviour of an object.
Syntax:

```
[modifiers ]<return Type><MethodName> (<parameter>)  
{  
    //Method implementation  
}
```

Type of Methods:

- I. Instance Methods
- II. Static Methods

I. Instance Method

- ❖ Method define inside the clas without using static modifier are called as Instance methods.
- ❖ You can access both instance and static members from instance methods.
- ❖ Instance method must be invoked with the reference variable which contains object.

II. Static Method

- ❖ Methods define inside the class with using static modifiere are called as Static methods.
- ❖ You can access only static members from static methods.
- ❖ Static method can invoked in three ways
 - ⇒ with class name
 - ⇒ with the reference variable which contains null.
 - ⇒ with the reference variable which contains object.

Example: -

class Hello { void m1(){ } static void m2() { } }	Hello h=new hello(); h.m1(); //VALID h.m2(); //VALID Hello.m1(); //INVALID Hello.m1(); //VALID Hello h=null; h.m1(); //INVALID h.m2(); //VALID
--	---

Programs:

OOPS141.java

```

class OOPS141{
    public static void main(String args[]){
        Hello h1=new Hello();
        h1.show();
    }
    class Hello {
        void show(){
            System.out.println("show ()");
        }
    }
}

```

OOPS142.java

```

class OOPS142{
    public static void main(String args[]){
        Hello. show();
    }
    class Hello{
        void show(){
            System.out.println("show ()");
        }
    }
}

```

OOPS143.java

```

class OOPS143{
    public static void main(String args[]){
        Hello h=null;
        h. show();
    }
    class Hello {
        void show() {
            System.out.println("show ()");
        }
    }
}

```

OOPS144.java

```

class OOPS144{
    public static void main(String args[]){
        Hello. show();
    }
    class Hello{
        static void show(){
            System.out.println("show ()");
        }
    }
}

```

OOPS145.java

```

class OOPS145{
    public static void main(String args[]){
        Hello h=null;
        h. show();
    }
    class Hello{
        static void show(){
            System.out.println("show ()");
        }
    }
}

```

OOPS146.java

```

class OOPS146{
    public static void main(String args[]){
        Hello h=new Hello();
        h. show();
    }
    class Hello{
        static void show(){
            System.out.println("show ()");
        }
    }
}

```

OOPS147.java

```

class OOPS147{
    public static void main(String args[]){
        Hello h=new Hello();
        h.show();
    }
}

```

```

class Hello{
    int a;
    static int b;
    void show() {
        System.out.println(a);
        System.out.println(b);
    }
}

```

OOPS148.java

```

class OOPS148{
    public static void main(String args[]){
        Hello. show();
    }
}
class Hello{
    int a;
    static int b;
    static void show() {
        System.out.println(a);
        System.out.println(b);
    }
}

```

OOPS149.java

```

class OOPS149{
    public static void main(String args[]){
        Hello. show();
    }
}
class Hello{
    static int b;
    static void show() {
        System.out.println(b);
    }
}

```

Program output /error and details:-

Prog No.	Output	Error and Details
OOPS141	show ()	Instance method must be invoked with the reference variable which contains object.
OOPS142	Compilation Error	[Error: Cannot make a static reference to the non-static method show() from the type Hello]
OOPS143	Runtime Error	[Error: java.lang.NullPointerException] because, Instance method cannot invoked with the reference variable which contains null .
OOPS144	show ()	Static method can invoked with class name
OOPS145	show ()	Static method can invoked with the reference variable which contains null.
OOPS146	show ()	Static method can invoked with the reference variable which contains object.
OOPS147	0 0	You can access both instance and static members from instance methods.
OOPS148	Compilation Error	[Error: Cannot make a static reference to the non-static field a] You cannot access instance members from static methods.
OOPS149	0	You can access static members from static methods.

Write your note here.....

Method Return type

- ❖ You are defining a method for performing an operation. After finishing operation it may produce some result.
- ❖ Result produced from method can be used in two ways
 - ⇒ Using the result inside the method.
 - ⇒ Returning the result to the caller method.
- ❖ When you want to return the result to the caller of the method then you must specify return type depending on the result.
- ❖ When you don't want to return the result to the caller of the method then you must specify return type as void.
- ❖ When return type is void then:
 - ⇒ You should not specify return statement.
 - ⇒ You can specify EMPTY RETURN statement.
- ❖ When return type is other than void then:
 - ⇒ You must specify return statement.
 - ⇒ You should not specify EMPTY RETURN statement.
- ❖ Return value must be same or compatible type to return type.

Programs:

OOPS150.java

```
class OOPS150{
    public static void main(String args[]){
        Hello h=new Hello();
        h.show();
    }
    class Hello{
        show(){
            System.out.println("Sai");
        }
    }
}
```

OOPS151.java

```
class OOPS151{
    public static void main(String args[]){
        Hello h=new Hello();
        h.setAge(-12);
        System.out.println(h.age);
    }
    class Hello{
        int age;
        void setAge(int age){
            this.age=age;
        }
    }
}
```

OOPS152.java

```
class OOPS152{
    public static void main(String args[]){
        Hello h=new Hello();
        h.setAge(-12);
        System.out.println(h.age);
    }
    class Hello{
        int age=18;
        void setAge(int age){
            if (age<18)
                return;
            this.age=age;
        }
    }
}
```

OOPS153.java

```
class OOPS153{
    public static void main(String args[]){
        Hello h=new Hello();
        h.setAge(-12);
        System.out.println(h.age);
    }
    class Hello{
        int age=18;
        void setAge(int age){
            if (age<18)
                return 0;
            this.age=age;
        }
    }
}
```

OOPS154.java

```
class OOPS154{  
    public static void main(String args[]){  
        Hello h=new Hello();  
        int a=h. show(10);  
        System.out.println(a);  
    }  
}  
class Hello{  
    int show(int x){  
        return x+1;  
    }  
}
```

OOPS155.java

```
class OOPS155{  
    public static void main(String args[]){  
        Hello h=new Hello();  
        System.out.println(h. show(10));  
    }  
}  
class Hello{  
    int show(int x){  
        return x+1;  
    }  
}
```

OOPS156.java

```
class OOPS156{  
    public static void main(String args[]){  
        Hello h=new Hello();  
        h. show(10);  
        System.out.println("Hello");  
    }  
}  
class Hello{  
    int show(int x){  
        return x+1;  
    }  
}
```

OOPS157.java

```
class OOPS157{  
    public static void main(String args[]){  
        Hello h=new Hello();  
        System.out.println(h.show(10));  
    }  
}  
class Hello{  
    void show(int x){  
        System.out.println("show()");  
    }  
}
```

OOPS158.java

```
class OOPS158{  
    public static void main(String args[]){  
        Hello h=new Hello();  
        h. show(10);  
        System.out.println("h");  
    }  
}  
class Hello{  
    int show(int x){  
        System.out.println("show()");  
        return 12L;  
    }  
}
```

OOPS159.java

```
class OOPS159{  
    public static void main(String args[]){  
        Hello h=new Hello();  
        h. show(10);  
        System.out.println("h");  
    }  
}  
class Hello{  
    int show(int x){  
        System.out.println("show()");  
        return;  
    }  
}
```

OOPS160.java

```
class OOPS160{  
    public static void main(String args[]){  
        Hello h=new Hello();  
        h. show(10);  
        System.out.println("Hello");  
    }  
}
```

```
class Hello{  
    int show(int x){  
        System.out.println("Show()");  
        return 12L;  
    }  
}
```

OOPS161.java

```
class OOPS161{
    public static void main(String args[]){
        Hello h=new Hello();
        System.out.println(h.show(10));
    }
}
```

```
class Hello{
    long show(int x){
        System.out.println("show()");
        return 'A';
    }
}
```

OOPS162.java

```
class OOPS162{
    public static void main(String args[]){
        Hello h=new Hello();
        System.out.println(h.show(10));
    }
    class Hello{
        long show(int x){
            System.out.println("show()");
            return x+1;
        }
    }
}
```

OOPS163.java

```
class OOPS163{
    public static void main(String args[]){
        Hello h=new Hello();
        System.out.println(h.show(10));
    }
    class Hello{
        boolean show(int x){
            System.out.println("show()");
            return true;
        }
    }
}
```

OOPS164.java

```
class OOPS164{
    public static void main(String args[]){
        Hello h=new Hello();
        System.out.println(h.show(10));
    }
    class Hello{
        boolean show(int x){
            System.out.println("show()");
            return 0;
        }
    }
}
```

OOPS165.java

```
class OOPS165{
    public static void main(String args[]){
        Hello h=new Hello();
        System.out.println(h.show(10));
    }
    class Hello{
        boolean show(int x){
            System.out.println("show()");
            return false;
            return false;
        }
    }
}
```

OOPS166.java

```
class OOPS166{
    public static void main(String args[]){
        Hello h=new Hello();
        System.out.println(h.isDigit('A'));
        System.out.println(h.isDigit('8'));
    }
    class Hello{
        boolean isDigit(char ch){
            System.out.println(ch+" is Digit ");
            boolean b=ch>=48 && ch<=57;
            return b;
        }
    }
}
```

OOPS167.java

```
class OOPS167{
    public static void main(String args[]){
        Hello h=new Hello();
        System.out.println(h.isDigit('A'));
        System.out.println(h.isDigit('8'));
    }
    class Hello{
        boolean isDigit(char ch){
            System.out.println(ch+" isDigit ");
            if (ch>=48 && ch<=57)
                return true;
            else
                return false;
        }
    }
}
```

Program output /error and details:-

Prog No.	Output	Error and Details
OOPS150	Compilation Error	[Error: invalid method declaration; return type required] because, you must have to use return type.
OOPS151	-12	When you don't want to return the result to the caller of the method then you must specify return type as void.
OOPS152	18	Here, return type is void then you should not specify return statement other wise use empty return.
OOPS153	Compilation Error	[Error: cannot return a value from method whose result type is void] if return type is void then use only empty return type.
OOPS154	1 1	When return type is other than void then you must specify return statement.
OOPS155	1 1	same"
OOPS156	Hello	Return type is valid but we are not printing show method.
OOPS157	Compilation Error	[Error: The method println (boolean) in the type Print Stream is not applicable for the arguments (void)] void type not allowed because you are passing integer type.
OOPS158	Compilation Error	[Error: Type mismatch: cannot convert from long to int] if return type is not void then specify return type.
OOPS159	Compilation Error	[Error: This method must return a result of type int] Because if return type is not void then you should not specify EMPTY RETURN statement.
OOPS160	Compilation Error	[Error: Type mismatch: cannot convert from long to int] Here, return type is integer so must use compatible return type.
OOPS161	show() 65	If return type is long then you can use return char type or int type.
OOPS162	show() 1 1	If return type is long then you can use return char type or int type.
OOPS163	show() true	If return type is boolean then you can use return boolean type only.
OOPS164	Compilation Error	[Error: Type mismatch: cannot convert from int to Boolean] If return type is boolean then you cannot use return type is other than boolean.
OOPS165	Compilation Error	[Error: Unreachable code] you cannot use two return type other wise use with condition.
OOPS166	A is Digit false 8 is Digit true	We are checking the condition if digit have ascii value between 48 to 57 then print true otherwise false.
OOPS167	A is Digit false 8 is Digit true	You may have multiple returns with condition.

Method Parameter

There are two type of argument:

1. Formal Argument
2. Actual Argument

1. Formal Argument

- ❖ Argument specified with the method definition or declaration is called as formal Argument.
- ❖ Formal Argument is also called as Parameter.

2. Actual Argument

- ❖ Argument specified with the method is called as Actual Argument.

Example:

```
class Hello {
    void show(int a, int b) {
        .....
    }
}
```

formal argument

```
Hello h=new Hello ();
h. show (10,20);
```

Actual Argument

- ❖ Actual and formal argument must be
 - ⇒ Same as terms of number of argument.
 - ⇒ Same or compatible in the terms of type of argument.
 - ⇒ Same in terms of order of argument.

Programs:

OOPS168.java

```
class OOPS168{
    public static void main(String args[]){
        Hello h=new Hello();
        System.out.println(h. show());
    }
}
class Hello{
    int show(int x){
        System.out.println("show()");
        return x+1;
    }
}
```

OOPS169.java

```
class OOPS169{
    public static void main(String args[]){
        Hello h=new Hello();
        System.out.println(h. show(10,20));
    }
}
class Hello{
    int show(int x){
        System.out.println("show()");
        return x+1;
    }
}
```

OOPS170.java

```
class OOPS170{
    public static void main(String args[]){
        Hello h=new Hello();
        h. show(20);
    }
}
```

```
}
class Hello{
    void show(byte x){
        System.out.println("show(byte)");
    }
}
```


OOPS171.java

```
class OOPS171{  
    public static void main(String args[]) {  
        Hello h=new Hello();  
        byte b=12;  
        h.show(b);  
    }  
}
```

```
class Hello{  
    void show(byte x){  
        System.out.println("show(byte)");  
    }  
}
```

OOPS172.java

```
class OOPS172{  
    public static void main(String args[]){  
        Hello h=new Hello();  
        h.show((byte)12);  
    }  
}  
class Hello{  
    void show(byte x){  
        System.out.println("show(byte)");  
    }  
}
```

OOPS173.java

```
class OOPS173{  
    public static void main(String args[]){  
        Hello h=new Hello();  
        h.show(65);  
    }  
}  
class Hello{  
    void show(char x){  
        System.out.println("show(char)");  
    }  
}
```

OOPS174.java

```
class OOPS174{  
    public static void main(String args[]){  
        Hello h=new Hello();  
        h.show('a');  
    }  
}  
class Hello{  
    void show(char x){  
        System.out.println("show(char)");  
    }  
}
```

OOPS175.java

```
class OOPS175{  
    public static void main(String args[]){  
        Hello h=new Hello();  
        h.show((char)65);  
    }  
}  
class Hello{  
    void show(char x){  
        System.out.println("show(char)");  
    }  
}
```

Write your note here.....

Program output /error and details:-

Prog No.	Output	Error and Details
OOPS168	Compilation Error	[Error: The method show(int) in the type Hello is not applicable for the arguments ()] Here, you must have to pass Actual argument as int.
OOPS169	Compilation Error	[Error: The method show(int) in the type Hello is not applicable for the arguments (int, int)] Because, Actual and formal argument must be same as terms of number of argument.
OOPS170	Compilation Error	[Error: The method show(byte) in the type Hello is not applicable for the arguments (int)]Here, Actual argument 12 is int type you must provide byte value.
OOPS171	show(byte)	Here, Actual and formal argument is same or compatible in the terms of type of argument.
OOPS172	show(byte)	Here, we Typecasting will happen so int 12 is convert into byte type.
OOPS173	Compilation Error	[Error: The method show(char) in the type Hello is not applicable for the arguments (int)] Here, Actual argument 65 is int type you must provide char value.
OOPS174	show(char)	Here, Actual and formal argument is same or compatible in the terms of type of argument.
OOPS175	show(char)	Here, we Typecasting will happen so 65 is convert into char type.

Write your note here.....

Method Overloading

You can write multiple methods inside the class with the same name by changing parameters. This process is called as METHOD OVERLOADING.

➤ When you are overloading then you must follow the following rules.

- ⇒ Method name must be same.
- ⇒ Method parameter must be changed in terms of
 - ⇒ Number of parameters
 - ⇒ Type of parameters
 - ⇒ Order of parameters
- ⇒ Method return type can be anything (same and different).

Example:

1. Hello class contains six method <pre>class Hello { void m1 (byte b) {...} //Invokes m1(byte b) void m1 (short s) {...} void m1 (int i) {...} void m1 (long a) {...} void m1 (float f) {...} void m1 (double d) {...} } Hello h=new Hello (); byte b=12; h. m1 (b);</pre>	2. Hello class contains five method <pre>class Hello { void m1 (short s) {...} //Invokes m1(short s) void m1 (int i) {...} void m1 (long a) {...} void m1 (float f) {...} void m1 (double d) {...} } Hello h=new Hello (); byte b=12; h. m1 (b);</pre>
3. Hello class contains four method <pre>class Hello { void m1 (int i) {...} //Invokes m1(int i) void m1 (long a) {...} void m1 (float f) {...} void m1 (double d) {...} } Hello h=new Hello (); byte b=12; h. m1 (b);</pre>	4. Hello class contains three method <pre>class Hello { void m1 (long a) {...} // Invokes m1(long l) void m1 (float f) {...} void m1 (double d) {...} } Hello h=new Hello (); byte b=12; h. m1 (b);</pre>
5. Hello class contains 2 method <pre>class Hello { void m1 (float f) {...} //Invoke m1 (float f) void m1 (double d) {...} } Hello h=new Hello (); byte b=12; h. m1 (b);</pre>	6. Hello class contains one method <pre>class Hello { void m1 (double d){..} //Invokes m1(double b) } Hello h=new Hello (); byte b=12; h. m1 (b);</pre>
7. Hello class contains no method <pre>class Hello { } Hello h=new Hello (); byte b=12; h. m1 (b);</pre>	

Programs:

OOPS176.java

```

class OOPS176{
    public static void main(String args[]){
        Hello h=new Hello();
        int a=h. add(10,20);
        System.out.println(a);
        h. add(10,20);
    }
    class Hello {
        int add (int a, int b){
            System.out.println("add(int, int)");
            return a+b;
        }
        void add (int a, int b){
            System.out.println("add(int, int)");
        }
    }
}

```

OOPS177.java

```

class OOPS177{
    public static void main(String args[]){
        Hello h=new Hello();
        int a=h. add (10, 20);
        System.out.println(a);
        int b=h. add (10, 20, 30);
        System.out.println(b);
    }
    class Hello {
        int add(int a, int b){
            System.out.println("add(int, int)");
            return a+b;
        }
        int add(int a, int b, int c){
            System.out.println("add(int, int, int)");
            return a+b+c;
        }
    }
}

```

OOPS178.java

```

class OOPS178{
    public static void main(String args[]){
        Hello h=new Hello();
        int a=h. add (10, 20);
        System.out.println(a);
        String b=h. add ("IND", 90);
        System.out.println(b);
    }
    class Hello {
        int add(int a, int b){
            System.out.println("add(int, int)");
            return a+b;
        }
        String add(String a, int b){
            System.out.println("add(str, int)");
            return a+b;
        }
    }
}

```

OOPS179.java

```

class OOPS179{
    public static void main(String args[]){
        Hello h=new Hello();
        String a=h. add (99, "Sai");
        System.out.println(a);
        String b=h. add ("Ram", 99);
        System.out.println(b);
    }
    class Hello {
        String add(int a, String b){
            System.out.println("add(int, Str)");
            return a+b;
        }
        String add(String a, int b) {
            System.out.println("add(str, int)");
            return a+b;
        }
    }
}

```

OOPS180.java

```

class OOPS180{
    public static void main(String args[]){
        Hello h=new Hello();
        byte b=20;
        h. add (10, b);
        h. add (b, b);
    }
    class Hello {
        void add(int a, byte b){
            System.out.println("add(int, byte)");
        }
    }
}

```

OOPS181.java

```

class OOPS181{
    public static void main(String args[]){
        Hello h=new Hello();
        byte b=20;
        h. add (b, 10);
        h. add (b, b);
    }
    class Hello {
        void add(byte a, int b){
            System.out.println("add(byte, int)");
        }
    }
}

```

OOPS182.java

```

class OOPS182{
    public static void main(String args[]){
        Hello h=new Hello();
        byte b=20;
        h.add(b, 10);
        h.add(10, b);
    }
    class Hello {
        void add(byte a, int b){
            System.out.println("add(byte, int)");
        }
        void add(int a, byte b){
            System.out.println("add(int, byte)");
        }
    }
}

```

OOPS183.java

```

class OOPS183{
    public static void main(String args[]){
        Hello h=new Hello();
        byte b=20;
        h.add(b, b);
    }
    class Hello {
        void add(byte a, int b){
            System.out.println("add(byte, int)");
        }
        void add(int a, byte b){
            System.out.println("add(int, byte)");
        }
    }
}

```

OOPS184.java

```

class OOPS184{
    public static void main(String args[]){
        Hello h=new Hello();
        h.show('A', 'B');
    }
    class Hello {
        void show(int a, int b){
            System.out.println("show(int, int)");
        }
        void show(long a, char b){
            System.out.println("show (long, char)");
        }
    }
}

```

OOPS185.java

```

class OOPS185{
    public static void main(String args[]){
        Hello h=new Hello();
        h.show('A', 'B');
    }
    class Hello {
        void show(int a, int b){
            System.out.println("show(int, int)");
        }
        void show(long a, long b){
            System.out.println("show (long, long)");
        }
    }
}

```

OOPS186.java

```

class OOPS186{
    public static void main(String args[]){
        Hello h=new Hello();
        h.show(null);
        h.show("Sai");
        h.show(h);
    }
    class Hello {
        void show(String str){
            System.out.println("show(String)");
        }
        void show(Object str){
            System.out.println("show (Object)");
        }
    }
}

```

OOPS187.java

```

class OOPS187{
    public static void main(String args[]){
        Hello h=new Hello();
        h.show("Sai");
        h.show(h);
    }
    class Hello {
        void show(String str){
            System.out.println("show(String)");
        }
        void show(Hello str){
            System.out.println("show(Hello)");
        }
    }
}

```

OOPS188.java

```

class OOPS188{
    public static void main(String args[]){
        Hello h=new Hello();
        h.show(null);
        h.show(h);
    }
    class Hello {
        void show(String str){
            System.out.println("show(String)");
        }
        void show(Hello str){
            System.out.println("show(Hello)");
        }
    }
}

```

OOPS189.java

```

class OOPS189{
    public static void main(String args[]){
        Hello h=new Hello();
        h.show(null);
    }
    class Hello {
        void show(String str){
            System.out.println("show(String)");
        }
        void show(char[] arr){
            System.out.println("show(char[])");
        }
    }
}

```

OOPS190.java

```

class OOPS190{
    public static void main(String args[]){
        System.out.println(10);
    }
}

```

OOPS191.java

```

class OOPS191{
    public static void main(String args[]){
        System.out.println(10L);
    }
}

```

OOPS192.java

```

class OOPS192{
    public static void main(String args[]){
        System.out.println(10, 20);
    }
}

```

OOPS193.java

```

class OOPS193{
    public static void main(String args[]){
        System.out.println("A", "B");
    }
}

```

OOPS194.java

```

class OOPS194{
    public static void main(String args[]){
        System.out.println('A');
    }
}

```

OOPS195.java

```

class OOPS195{
    public static void main(String args[]){
        System.out.println(null);
    }
}

```

Program output /error and details:-

Prog No.	Output	Error and Details
OOPS176	Compilation Error	[Error: method add(int, int) is already defined in class Hello] Here, Method name and return type must be same.
OOPS177	add(int, int) 30 add(int, int, int) 60	Method parameter must be changed in terms of Number of parameters.

Prog No.	Output	Error and Details
OOPS178	add(int, int) 30 add(str, int) IND90	Method parameter must be changed in terms of type of parameters.
OOPS179	add(int, Str) 99 Sai add(str, int) Ram 99	Method parameter must be changed in terms of order of parameters.
OOPS180	add(int, byte) add(int, byte)	Actual argument (10, b) is int and byte type but you can also store byte value in int type.
OOPS181	add(byte, int) add(byte, int)	same””
OOPS182	add(byte, int) add(int, byte)	Method parameter must be changed in terms of order of parameters.
OOPS183	Compilation Error	[Error: The method add(byte, int) is ambiguous for the type Hello] Here, Actual argument (b, b) both are same type so compiler get confuse which value will call
OOPS184	Compilation Error	[Error: The method add(byte, int) is ambiguous for the type Hello] Here, if formal argument (2 nd) is same type then compiler not giving error.
OOPS185	show(int, int)	First lower type will be referring then higher type.
OOPS186	show(String) show(String) show (Object)	Same””. Here, object is super class.
OOPS187	show(String) show(Hello)	Here, Class Hello and String have same priority but actual argument is String type.
OOPS188	Compilation Error	[Error: The method show (String) is ambiguous for the type Hello] actual argument is null and compiler will not able to identify it string or other type.
OOPS189	Compilation Error	[Error: The method show(String) is ambiguous for the type Hello]
OOPS190	10	You can print int value directly.
OOPS191	10	You can print int value directly.
OOPS192	Compilation Error	[Error: The method println (int) in the type Print Stream is not applicable for the arguments (int, int)] print method is not define with 2 parameter.
OOPS193	Compilation Error	[Error: The method println (int) in the type Print Stream is not applicable for the arguments (String, String)] print method is not define with 2 parameter.
OOPS194	A	It is valid.
OOPS195	Compilation Error	[Error: The method println (char []) is ambiguous for the type Print Stream] actual argument is null and compiler will not able to identify it string or other type.

Recursion

- ⇒ Recursion is the process of calling one method from itself.
- ⇒ Primary memory only use for execution.
- ⇒ Variable memory will be removed is called de-allocation of memory.

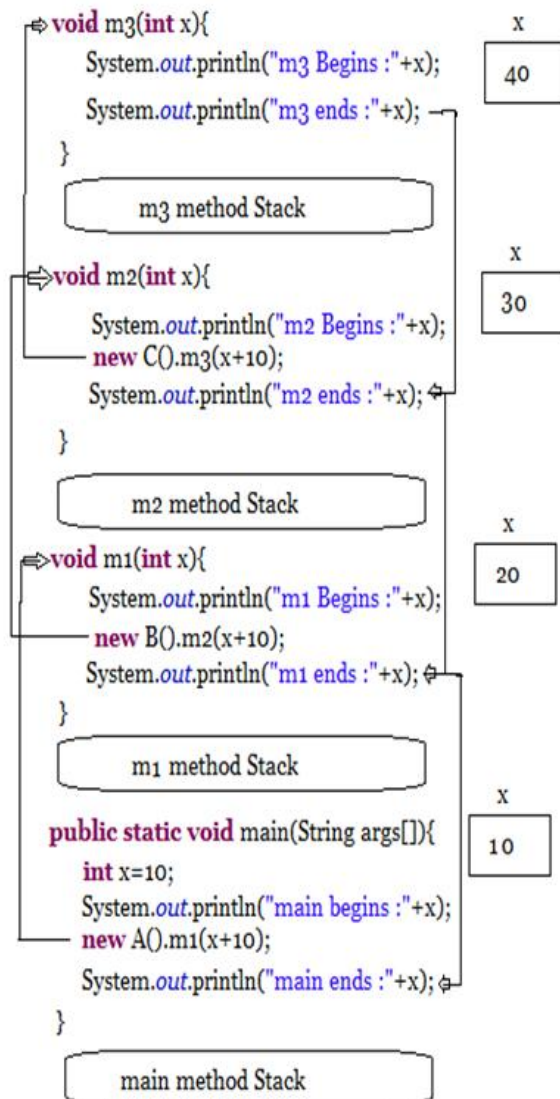
OOPS196.java

```
class C {
    void m3(int x){
        System.out.println("m3 Begins :"+x);
        System.out.println("m3 ends :"+x);
    }
}
class B{
    void m2(int x){
        System.out.println("m2 Begins :"+x);
        new C().m3(x+10);
        System.out.println("m2 ends :"+x);
    }
}
class A{
    void m1(int x){
        System.out.println("m1 Begins :"+x);
        new B().m2(x+10);
        System.out.println("m1 ends :"+x);
    }
}
class OOPS196{
    public static void main(String args[]){
        int x=10;
        System.out.println("main begins :"+x);
        new A().m1(x+10);
        System.out.println("main ends :"+x);
    }
}
```

OUTPUT

```
main begins :10
m1 Begins :20
m2 Begins :30
m3 Begins :40
m3 ends :40
m2 ends :30
m1 ends :20
main ends :10
```

STACK MEMORY



Details:

First main method Stack is creating then main begins statement will execute then m1 method will call then m1 begins statement will execute.... m3 method then memory de-allocated and order of invocation will m3,m2,m1and main and all ends statement will execute with order of invocation .

OOPS197.java

```

class OOPS197{
public static void main(String
args[]){
int x=98;
System.out.println("main begins
:"+x);
new Hello().m1(x);
System.out.println("main Ends :"+x);
}
}
class Hello{
void m1(int x){
System.out.println("m1 Begins
:"+x);
if(x!=0)
m1(x/10);
System.out.println("m1 Ends
:"+x);
}
}

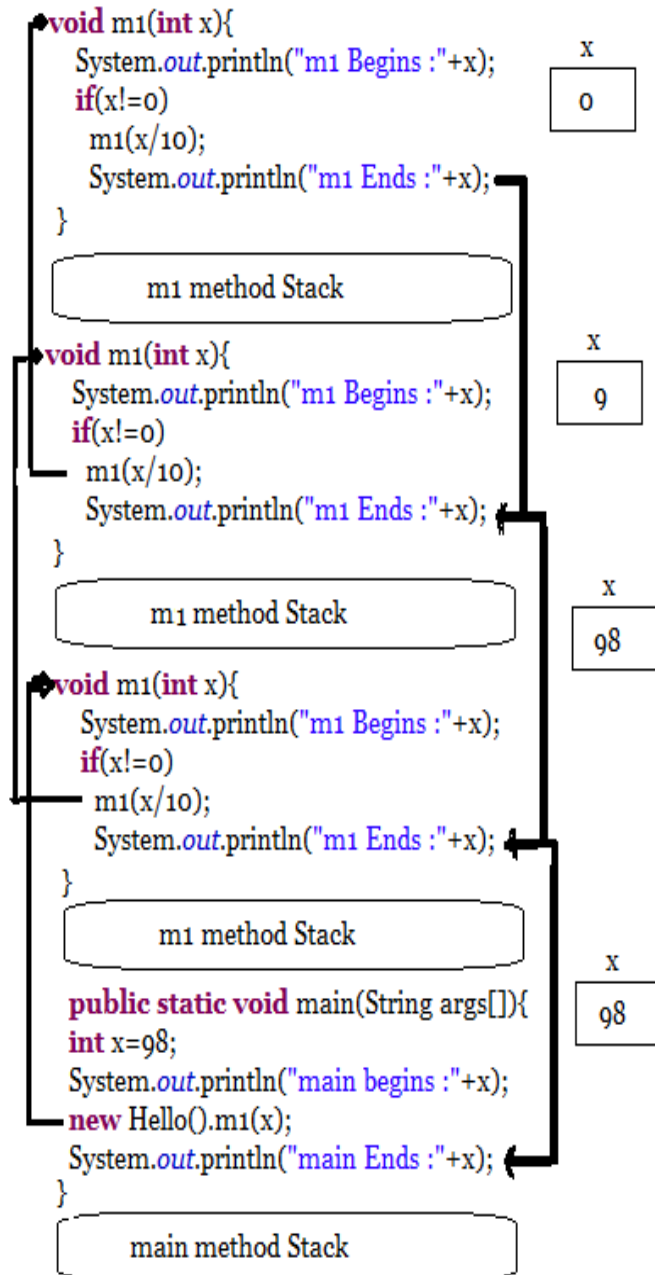
```

OUTPUT

```

main begins :98
m1 Begins :98
m1 Begins :9
m1 Begins :0
m1 Ends :0
m1 Ends :9
m1 Ends :98
main Ends :98

```

Stack memory**Details:**

First main method Stack is creating then main begins statement will execute then m1 method will call then m1 begins statement will execute then check the condition if true then m1 stack will create and check the condition every time when value is false then all the end statement will execute then memory de-allocated in order of invocation.

OOPS198.java

```
class OOPS198{
public static void main(String args[]){
Hello h=new Hello();
int n=4;
long fac=h. factorial(n);
System.out.println("Factorial is :"+fac);
}
}
class Hello{
long factorial(int n){
if(n==0||n==1)
return 1;
else
return n*factorial(n-1);
}
}
```

OUTPUT

Factorial is :24

Details: - Same as OOPS197.java

OOPS199.java

```
class OOPS199{
public static void main(String args[]){
Hello h=new Hello();
int n=3;
int p=2;
System.out.println("Power is :
"+h.power(n,p));
}
}
class Hello{
long power(int n,int p){
if(p==0)
return 1;
else
return n*power(n,p-1);
}
}
```

OUTPUT

Power is : 9

Details: - Same as OOPS197.java

Call by Value / Call by Reference

Call by Value:

- ❖ When you invoke a method by passing primitive data type then it is called as Call by Value.
- ❖ In the case of call by value, modifications happened inside the called method will not be reflected to caller method.

Call by Reference:

- ❖ (High level java program can't copy address so only low level java program can support.)
- ❖ When you invoke a method by passing reference data type then it is called as Call by Reference.
In case of call by reference,
- ❖ When we modify property of the object then modifications happened inside the called method will be reflected to caller method.
- ❖ When we modify reference itself then modifications happened inside the called method will not be reflected to caller method.

OOPS200.java

```
class OOPS200{
public static void main(String args[]){
int a=99;
Hello h=new Hello();
System.out.println("main begin :"+a);
h.m1(a);
System.out.println("main end :"+a);
}
}
class Hello{
void m1(int a){
System.out.println("m1 begin :"+a);
a=a+10;
System.out.println("m1 end :"+a);
}
}
```

OUTPUT

```
main begin :99
m1 begin :99
m1 end :109
main end :99
```

Details:-

This program for call by value, modifications happened inside the called method will not be reflected to caller method.

OOPS201.java

```
class OOPS201{
public static void main(String args[]){
Hai hai=new Hai();
hai.a=99;
Hello hello=new Hello();
System.out.println("main begin :"+hai.a);
hello.m1(hai);
System.out.println("main end :"+hai.a);
}
}
class Hello{
void m1(Hai hai){
System.out.println("m1 begin :"+hai.a);
hai.a=hai.a+10;
System.out.println("m1 end :"+hai.a);
}
}
class Hai{ int a; }
```

OUTPUT

```
main begin :99
m1 begin :99
m1 end :109
main end :109
```

Details:-

This program for call by reference, modify property of the object then modifications happened inside the called method will be reflected to caller method so that value is changing.

OOPS202.java

```
class OOPS202{
public static void main(String args[]){
Hai hai=new Hai();
hai.a=99;
Hello hello=new Hello();
System.out.println("main begin :"+hai.a);
hello.m1(hai);
System.out.println("main end :"+hai.a);
}
}
class Hello{
void m1(Hai hai){
System.out.println("m1 begin :"+hai.a);
hai=new Hai();
hai.a=hai.a+10;
System.out.println("m1 end :"+hai.a);
}
}
class Hai{
int a;
}
```

OUTPUT

```
main begin :99
m1 begin :99
m1 end :10
main end :99
```

OOPS203.java

```
class OOPS203{
public static void main(String args[]){
Hai hai=new Hai();
hai.a=99;
Hello hello=new Hello();
System.out.println("main begin :"+hai.a);
hello.m1(hai.a);
System.out.println("main end :"+hai.a);
}
}
class Hello{
void m1(int a){
System.out.println("m1 begin :"+a);
a=a+10;
System.out.println("m1 end :"+a);
}
}
class Hai{
int a;
}
```

OUTPUT

```
main begin :99
m1 begin :99
m1 end :109
main end :99
```

Details: do

Var-Arg:

- ❖ It is added from java 5.
- ❖ When you want to define a method to take multiple values of specific type then you can use the following option.
 - ⇒ Overload the method.
 - ⇒ Use array as parameters.

Example: -

```
class Hello {  
    void sum(int a, int b) {...}  
    void sum(int a, int b, int c) {...}  
    void sum(int a, int b, int c, int d) {...}  
    .....  
}  
  
class Hello{  
    void sum(int [] arr) {...}  
}
```

(Or)

- ❖ The above said approaches having some complexity. To avoid the complexity VAR-ARGS concept has introduced.
- ❖ With variable arguments concept, you can define a method which can take any number of arguments of the same type.

Syntax: -

```
<return Type> <method Name> (<data Type> <FixedArg>, <data Type> ... <VarArg>)  
{  
    .....  
}
```

Example: -

```
void sum (int... values){....}  
void sum (String str, int ... values) {....}
```

- ❖ To define Var-Args parameter. you need to use ellipsis [three ... (dot)] after data type followed by variable name.

Example:-

```
void sum(int... values)
```

- ❖ Var-Args parameter will be converted to an array by the java compiler while generating class file.

Example:-

```
void sum(int... values)    -> void sum (int[] values)
```

- ❖ When you are calling a method which contains Var-Args parameter by passing zero or more values then java compiler converts it to an array

Example:-

```
sum () ;                -> sum (new int[0]);  
sum (99,88) ;           -> sum (new int[] {99,88});  
sum (99,88,77) ;        -> sum (new int[] {99,88,77});
```

- ❖ Var-Args can be used only to define parameters of method or constructor and cannot be use for instance variable, static variable and local variable.

- ❖ Var-Args parameter is equivalent to an array. So data available in Var-Args parameter can be accessed like array.

Example:-

```
void sum(int... values) {
    for (int i=0; i<values.length; i++)
        System.out.println (values[i]);
}
```

- ❖ You can pass actual values or array object to the method which is having Var-Args.

Example:-

```
sum(10, 20)
int arr[]=new int []{10,20};
sum (arr);
```

Programs:

OOPS204.java

```
class OOPS204{
    public static void main(String args[]){
        Hello h=new Hello();
        h. sum(12, 23);
    }
    class Hello{
        void sum(int a, int b){
            System.out.println ("** sum (int, int) :");
            System.out.println (a+b);
        }
    }
}
```

OOPS205.java

```
class OOPS205{
    public static void main(String args[]){
        Hello h=new Hello();
        h. sum(12, 32);
        h. sum(12, 32, 43);
    }
    class Hello{
        void sum(int a, int b){
            System.out.println ("** sum (int, int) :");
            System.out.println (a+b);
        }
        void sum(int a, int b, int c){
            System.out.println ("** sum (int, int, int) :");
            System.out.println (a+b+c);
        }
    }
}
```

OOPS206.java

```
class OOPS206{
    public static void main(String args[]){
        Hello h=new Hello();
        h. sum(new int[o]);
        h. sum(new int[]{12, 34, 54});
        h. sum(new int[]{10, 20, 50, 30});
    }
    class Hello{
        void sum(int arr[]){
            System.out.println ("** sum (int[]) :");
            System.out.println ("Length:"+arr.length);
            int s=0;
            for(int a:arr){
                s=s+a;
            }
            System.out.println ("sum is :"+s);
        }
    }
}
```

OOPS207.java

```
class OOPS207{
    public static void main(String args[]){
        Hello h=new Hello();
        h. sum();
        h. sum(12, 34, 54);
        h. sum(10, 20, 50, 30);
    }
    class Hello{
        void sum(int arr[]){
            System.out.println ("**show (int[]) :");
            System.out.println ("Length:"+arr.length);
            int s=0;
            for(int a:arr) {
                s=s+a;
            }
            System.out.println ("sum is :"+s);
        }
    }
}
```

OOPS208.java

```

class OOPS208{
    public static void main(String args[]){
        Hello h=new Hello();
        h.sum();
        h.sum(12, 34, 54);
        h.sum(10, 20, 50, 30);
    }
}
class Hello{
    void sum(int... arr){
        System.out.println ("**show (int...) ");
        System.out.println ("Length: "+arr.
length);
        int s=0;
        for(int a: arr){
            s=s+a;
        }
        System.out.println ("sum is : "+s);
    }
}

```

OOPS209.java

```

class OOPS209{
    public static void main(String args[]){
        Hello h=new Hello();
        h.sum(new int[o]);
        h.sum(new int[]{12, 34, 54});
        h.sum(new int[]{10, 20, 50, 30});
    }
}
class Hello{
    void sum(int... arr){
        System.out.println ("**show (int...) ");
        System.out.println ("Length: "+arr. length);
        int s=0;
        for(int a: arr){
            s=s+a;
        }
        System.out.println ("sum is : "+s);
    }
}

```

OOPS210.java

```

class OOPS210{
    public static void main(String args[]){
        Hello h=new Hello();
        h.show();
    }
}
class Hello{
    void show(int a, int... arr){
        System.out.println ("**show (int...) ");
    }
}

```

OOPS211.java

```

class OOPS211{
    public static void main(String args[]){
        Hello h=new Hello();
        h.show(10);
        h.show(10, 20);
        h.show(10, 20, 30);
    }
}
class Hello{
    void show(int a, int... arr){
        System.out.println ("**show (int...) ");
    }
}

```

OOPS212.java

```

class OOPS212{
    public static void main(String args[]){
        Hello h=new Hello();
        h.m1();
    }
}
class Hello{
    void m1(int... a){
        System.out.println ("m1(int...) ");
    }
    void m1(String... a){
        System.out.println ("m1(String...) ");
    }
}

```

OOPS213.java

```

class OOPS213{
    public static void main(String args[]){
        Hello h=new Hello();
        h.show(10);
    }
}
class Hello{
    void show(int arr1, int... arr2){
        System.out.println ("m1(int arr1, int...) ");
    }
}

```

OOPS214.java

```

class OOPS214{
    public static void main(String args[]){
        Hello h=new Hello();
        h.show(10);
    }
    class Hello{
        void show(int... arr1){
            System.out.println ("show (int...) ");
        }
        void show(int[] arr1){
            System.out.println ("show (int...) ");
        }
    }
}

```

OOPS215.java

```

class OOPS215{
    public static void main(String args[]){
        Hello h=new Hello();
        h.show(10, "Sai");
    }
    class Hello{
        void show(int... arr1, String str){
            System.out.println ("show (int... , String) ");
        }
    }
}

```

OOPS216.java

```

class OOPS216{
    public static void main(String args[]){
        Hello h=new Hello();
        h.show("Sai", 10);
    }
    class Hello{
        void show(String str, int... arr1){
            System.out.println ("show (String, int...)");
        }
    }
}

```

OOPS217.java

```

class OOPS217{
    public static void main(String args[]){
        Hello h=new Hello();
        h.show("Sai", 10);
    }
    class Hello{
        void show(String... str, int... arr1) {
            System.out.println ("show (String, int...)");
        }
    }
}

```

OOPS218.java

```

class OOPS218{
    public static void main(String args[]){
        Hello h=new Hello();
        h.show (10);
    }
    class Hello{
        void show(int[]... arr) {
            System.out.println ("show (int[]...) ");
        }
    }
}

```

OOPS219.java

```

class OOPS219{
    public static void main(String args[]){
        Hello h=new Hello();
        h.show();
    }
    class Hello{
        void show(int[]... arr) {
            System.out.println ("show (int[]...) ");
        }
    }
}

```

OOPS220.java

```

class OOPS220{
    public static void main(String args[]){
        Hello h=new Hello();
        int arr1[]=new int[]{10};
        int arr2[]=new int[]{20, 30, 12};
        h.show(arr1, arr2);
    }
}

```

```

class Hello{
    void show(int[]... arr){
        System.out.println ("show (int[]...) ");
    }
    for(int[] tmp: arr){
        for(int a: tmp)
            System.out.println (a+ " ");
        System.out.println ();
    }
}

```


OOPS221.java

```
class OOPS221{
    public static void main(String args[]){
        Hello h=new Hello();
        int arr1[]=new int[]{10};
        int arr2[]=new int[]{20, 30, 12};
        h.show(arr1, arr2);
    }
}
```

```
class Hello{
    void show(int... arr){
        System.out.println ("show(int[...]) ");
    }
    for(int[] tmp: arr)    {
        for(int a: tmp)
            System.out.println (a+ " ");
        System.out.println ();
    }
}
```

OOPS222.java

```
class OOPS222{
    public static void main(String args[]){
        Hello h=new Hello();
        h.show(10, 20);
    }
    class Hello{
        void show(int a, int b){
            System.out.println ("show(int, int) ");
        }
    }
}
```

OOPS223.java

```
class OOPS223{
    public static void main(String args[]){
        Hello h=new Hello();
        h.show(10, 20);
    }
    class Hello{
        void show(int... b){
            System.out.println ("show (int...) ");
        }
        void show(int a, int b){
            System.out.println ("show(int, int) ");
        }
    }
}
```

OOPS224.java

```
class OOPS224{
    public static void main(String args[]){
        Hello h=new Hello();
        h.show(10, 20);
    }
    class Hello{
        void show(int... b){
            System.out.println ("show (int...) ");
        }
        void show(int a, int b){
            System.out.println ("m1 (int, int) ");
        }
    }
}
```

OOPS225.java

```
class OOPS225{
    public static void main(String args[]){
        Hello h=new Hello();
        h.show(10, 20);
    }
    class Hello{
        void show(int... b){
            System.out.println ("show (int...) ");
        }
        void show(long a, long b){
            System.out.println ("show(long, long) ");
        }
    }
}
```

Program output /error and details:-

Prog No.	Output	Error and Details
OOPS204	** sum (int, int) : 35	Here, you are passing formal and actual argument and both are same and performing addition operation
OOPS205	** sum (int, int) : 44 ** sum (int, int, int): 87	Here, you are passing 2 and 3 formal and actual argument and overload sum method and performing addition operation.

Prog No.	Output	Error and Details
OOPS206	<pre> ** sum (int[]) : Length:0 sum is :0 ** sum (int[]) : Length:3 sum is :100 ** sum (int[]) : Length:4 sum is :110 </pre>	Here, Defining method for multiple value and so that we have to overload the sum method for that we are using array as parameters. 1st time value is 0 so length is 0 2nd time value is 3 so length is 3 3rd time value is 4 so length is 4
OOPS207	Compilation Error	[Error: The method sum(int[]) in the type Hello is not applicable for the arguments ()] Here, you must have to array as parameter.
OOPS208	<pre> **show (int...) Length: 0 sum is : 0 **show (int...) Length: 3 sum is : 100 **show (int...) Length: 4 sum is : 110 </pre>	With variable arguments concept, you can define a method which can take any number of arguments of the same type and also if we are not passing any value it will take value as zero.
OOPS209	<pre> **show (int...) Length: 0 sum is : 0 **show (int...) Length: 3 sum is : 100 **show (int...) Length: 4 sum is : 110 </pre>	Var-Args parameter will be converted to an array by the java compiler while generating class file. so that you can use array as parameter to pass the value.
OOPS210	Compilation Error	[Error: The method show (int, int...) in the type Hello is not applicable for the arguments ()] you are not passing any argument.
OOPS211	<pre> **show (int...) **show (int...) **show (int...) </pre>	When you are calling a method which contains Var-Args parameter by passing zero or more values then java compiler converts it to an array. Here, We are passing 3 time argument and calling show method.
OOPS212	Compilation Error	[Error: The method m1(int[]) is ambiguous for the type Hello] Here, we are not passing any value and overloading the m1 method with int and string type value so it gives error because compiler not able to identify value is int type or string type.
OOPS213	m1 (int arr1, int...)	we can pass actual argument one and formal argument as Var-Args more than one.
OOPS214	Compilation Error	[Error: cannot declare both show(int[]) and show(int...) in Hello] Here, Show method is not overload because both are same type.

Prog No.	Output	Error and Details
OOPS215	Compilation Error	[Error: ')' expected] Var-Args is always last argument
OOPS216	show (String, int...)	We can pass two different argument.
OOPS217	Compilation Error	[Error: ')' expected] we can't pass two Var-Args.
OOPS218	Compilation Error	[Error: The method show(int[]...) in the type Hello is not applicable for the arguments (int)] Here, required: int[][] found: int reason: argument type int does not conform to Var-Arg element type int[]
OOPS219	show (int[]...)	We can pass empty argument type.
OOPS220	Compilation Error	[Error: illegal start of type [for(int[] tem: arr)]
OOPS221	Compilation Error	[Error: illegal start of type [for(int[] tem: arr)]
OOPS222	show(int, int)	Here, Actual and formal argument both are same.
OOPS223	show(int, int)	You can pass Actual argument more than one but same type when formal argument is Var-Args.
OOPS224	m1 (int, int)	First it refer the hard coded argument.
OOPS225	show(long, long)	same""

Introduction to Modifiers

Modifiers	Top Level Class	class Variable	Local Variable	class Block	Local Block	Constructor	Method
Private	NA	A	NA	NA	NA	A	A
Protected	NA	A	NA	NA	NA	A	A
Public	A	A	NA	NA	NA	A	A
final	A	A	A	NA	NA	NA	A
static	NA	A	NA	A	NA	NA	A
abstract	A	NA	NA	NA	NA	NA	A
native	NA	NA	NA	NA	NA	NA	A
strictfp	A	NA	NA	NA	NA	NA	A
Synchronized	NA	NA	NA	NA	A	NA	A
transient	NA	A	NA	NA	NA	NA	NA
volatile	NA	A	NA	NA	NA	NA	NA

Here, A: - Allowed, NA: - Not Allowed

Main Method

- ❖ main() method is a special method which will be called by the JVM.
- ❖ main() method has the standard signature as follows.
 - ⇒ `public static void main ( String[] args) {}`
 - ⇒ `static public void main ( String[] args) {}`
- ❖ You can overload main() method but JVM always invokes main() method with standard signature.
- ❖ main() method works as application launcher i.e. it is starting point of your application.

Explanation of main() method:

public	It can be accessed from anywhere.
static	It can be accessed using Class Name directly.
void	It will not return any value to caller (JVM).
main	Method Name (Taken from c/c++)
String[]	To pass command line values.

JVM task before java7:

- ⇒ Loads the class.
- ⇒ It checks whether main() method with standard signature available or not.
- ⇒ If available then invokes the main() method.
- ⇒ If not available then JVM will throw “java.lang.NoSuchMethodError:main” error.

JVM task java7 onwards:

- ⇒ It checks whether main () method with standard signature available or not.
- ⇒ If available then loads the class first and invokes main () method.
- ⇒ If not available then JVM will throw Error.

“Main method not found in class Hello, Please define the main method as:

`public static void main ( String[] args) {}”.`

- ❖ You can use Var-Args as main () method parameter.
- ❖ You can use following modifiers for standard main() method.
 - public
 - static
 - final
 - strictfp
 - synchronized
- ❖ You cannot use following modifiers for standard main() method.
 - abstract
 - native
 - private
 - protected
- ❖ You can invoke main() method explicitly.

Programs:

OOPS226.java

```
class OOPS226{
    static{
        System.out.println("Static block");
    }
    public static void main(String args){
        System.out.println("main()");
    }
}
```

OOPS227.java

```
class OOPS227{
    static{
        System.out.println("static block");
    }
    public static void main(String[] args){
        System.out.println("main()");
    }
}
```

OOPS228.java

```
class OOPS228{
    public static void main(String args[]){
        System.out.println("main()");
    }
}
```

OOPS229.java

```
class OOPS229{
    public static void main(String... args){
        System.out.println("main()");
    }
}
```

OOPS230.java

```
class OOPS230{
    final public static void main(String args[]){
        System.out.println("main()");
    }
}
```

OOPS231.java

```
class OOPS231{
    strictfp public static void main(String args[]){
        System.out.println("main()");
    }
}
```

OOPS232.java

```
class OOPS232{
    synchronized public static void ain(String
args[]){
        System.out.println("main()");
    }
}
```

OOPS233.java

```
class OOPS233{
    private static void main(String... args){
        System.out.println("main()");
    }
}
```

OOPS234.java

```
class OOPS234{
    public void main(String args[ ]){
        System.out.println("main()");
    }
}
```

OOPS235.java

```
class OOPS235{
    public static abstract void main(String... args){
        System.out.println("main()");
    }
}
```

OOPS236.java

```
class OOPS236{
    native static public void main(String... arg){
        System.out.println("main()");
    }
}
```

OOPS237.java

```
class OOPS237{
    public static final synchronized strictfp
void main(String... args){
        System.out.println("main()");
    }
}
```

OOPS238.java

```
class OOPS238{
    public static void main(String args){
        System.out.println("main(String)");
    }
    public static void main(String[] args){
        System.out.println("main(String[])");
    }
    public static void main(int[] args){
        System.out.println("main(int[])");
    }
}
```

OOPS239.java

```
class OOPS239{
    public static void main(String args[]){
        System.out.println("main(String)");
    }
    public static void main(int[] args){
        System.out.println("main(int[])");
    }
}
```

OOPS240.java

```
class OOPS240{
    public static void main(String args[]){
        System.out.println("main Start");
        main (null);
    }
}
```

OOPS241.java

```
class OOPS241{
    static int count=0;
    public static void main(String[] args){
        count++;
        System.out.println("main Start: "+count);
        if(count<3)
            main (null);
        System.out.println("main ends: "+count);
    }
}
```

OOPS242.java

```
class OOPS242{
    public static void main(String args[]){
        System.out.println(args);
    }
    public static void main(String[] args){
        System.out.println("main (string[])");
        main("SAI");
        main("SAI");
    }
}
```

OOPS243.java

```
class OOPS243 {
    public static void main(String[] args){
        System.out.println("Hello main");
    }
}
class Hai {
    public static void main(String[] args){
        System.out.println("Hai main");
    }
}
```

OOPS244.java

```
class OOPS244 {
{
    System.out.println("OOPS244-IB");
}
Java774(){
    System.out.println("OOPS244-DC");
}
public static void main(String[] args){
    System.out.println(" main");
}
}
```

OOPS245.java

```
class OOPS245 {
    public static void main(String[] args){
        System.out.println("Hello main");
    }
}
class Hai{
    public static void main(String[] args){
        System.out.println("Hai main");
        Hello. main(args);
    }
}
```

OOPS246.java

```

class OOPS246 {
    static {
        System.out.println("Static block");
        main(new String[]{"SAI", "RAM"});
    }

    public static void main(String... args){
        System.out.println("main():"+args.length);
        for(String str: args)
            System.out.println(str);
    }
}

```

OOPS247.java

```

class OOPS247 {
    System.out.println("Instance block");
    main(new String[]{"SAI", "RAM"});
}

public static void main(String... args){
    System.out.println (" main(): "+args.length);
    for(String str: args)
        System.out.println(str);
}
}

```

Program output /error and details:-

Prog No.	Output	Error and Details
OOPS226	Runtime Error	[Error: Main method not found in class Java756, please define the main method as: public static void main(String[] args)] missing args[]
OOPS227	static block main()	1 st load the class and execute static block then main method executed.
OOPS228	main()	It is valid main() method standard signature.
OOPS229	main()	You can use Var-Args as main () method parameter.
OOPS230	main()	You can use final modifiers for standard main() method.
OOPS231	main()	You can use Strictfp modifiers for standard main() method.
OOPS232	main()	You can use synchronized modifiers for standard main() method.
OOPS233	Runtime Error	[Error: Main method not found in class Java763, please define the main method as: public static void main(String[] args)] main method not private.
OOPS234	Runtime Error	[Error: Main method is not static in class Java764, please define the main method as: public static void main(String[] args)]
OOPS235	Compilation Error	[Error: illegal combination of modifiers]
OOPS236	Compilation Error	[Error: illegal combination of modifiers]
OOPS237	main()	You can use this combination of modifiers.
OOPS238	main(String[])	You can overload main() method but JVM always invokes main() method with standard signature.
OOPS239	Runtime Error	[Error: Main method not found in class Java769, please define the main method as: public static void main(String[] args)] missing String [] args.

Prog No.	Output	Error and Details
OOPS240	Runtime Error	[Error: main Start main Start main Start main Start Stack Overflow]
OOPS241	main Start: 1 main Start: 2 main Start: 3 main ends: 3 main ends: 3 main ends: 3	first memory allocated for static variable with 0 value then main method executed then value of variable will increase by one and check the condition if true then go to the Main end statement.
OOPS242	main (string[]) SAI SAI	If inside the have two main method with same standard signature of main method then only one method will loaded.
OOPS243	Hello main	If two classes have same standard signature then only one class will load those are define as .java file.
OOPS244	main	Instance block will not execute because object is not creating here.
OOPS245	Hello main	If two classes have same standard signature then only one class will load those are first class.
OOPS246	Static block main() : 2 SAI RAM main() : 0	first static block will executed then main method and return length 2 because two argument is passing. str value is zero due to default value.
OOPS247	main(): 0	instance block will not execute.

Write your note here.....